Roveda, L., Maskani, J., Franceschi, P., Abdi, A., Braghin, F., Molinari Tosatti, L., and Pedrocchi, N. (2020). Model-based reinforcement learning variable impedance control for human-robot collaboration. *J. of Int. Rob. Sys.*, 100(2), 417–433.

Schoettler, G., Nair, A., Luo, J., Bahl, S., Ojea, J.A., Solowjow, E., and Levine, S. (2020). Deep reinforcement learning for industrial insertion tasks with visual inputs and natural rewards. In *Int. Conf. on Int. Rob. and Sys(IROS)*, 5548–5555. IEEE.

Shoemake, K. (1985). Animating rotation with quaternion curves. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques*, 245–254. Association for Computing Machinery, New York, NY, USA.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T., Simonyan, K., and Hassabis, D. (2018a). A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419), 1140–1144.

Silver, T., Allen, K., Tenenbaum, J., and Kaelbling, L. (2018b). Residual policy learning. *arXiv preprint arXiv:1812.06298*.

Todorov, E., Erez, T., and Tassa, Y. (2012). Mujoco: A physics engine for model-based control. In *IEEE Int. Conf. on Int. Rob. and Sys(IROS)*, 5026–5033. IEEE.

Appendix A. EXPERIMENTAL DETAILS

This section gives further details on the polishing environment and the nominal controller, as well as hyperparameters for all experiments.

A.1 Derivation of the Task Goal

The goal of our task is to polish over the workpiece with a constant material removal rate (MRR). For this, we let a robotic arm with a polishing tool attached to the last joint follow a predefined path r^{path} over the bridge workpiece. To refine the requirements, further insights about the definition of MRR can be derived for our task setup.

The normal pressure p_N is defined as the normal force applied to a certain contact region of the surface $A \in \mathbb{R}$. In the case of low friction, the normed force is similar to the normal force F_N, as tangential forces can be omitted. When a spherical workpiece has contact with a plane polishing tool, the contact region has the geometry of an ellipsis. The size and shape of the ellipsis depend on the curvature of the workpiece, the applied force, and some material constants. Due to small expected variations in contact force, we approximate the area of contact to be constant. For this assumption to be valid, we require alignment between tool orientation and workpiece surface normal.

When the tool is moving along the path r^{path}, the relative velocity of the tool v_{rel} can be split into two parts: translational velocity $v_T \in \mathbb{R}$ and rotational velocity $v_B \in \mathbb{R}$. As we use a non-rotating polishing tool, we assume $v_B = 0$. The translational velocity v_T depicts the movement along the path.

Given these insights, we can calculate the material removal rate as

$$\begin{aligned}\frac{dh}{dt} &= K_p \cdot p_N \cdot v_{rel} \\ &= K_p \cdot \frac{F_N}{A} \cdot (v_B + v_T) \\ &= \frac{K_p}{A} \cdot F_N \cdot v_T\end{aligned}$$

The term $\frac{K_p}{A}$ is a constant factor. A constant MRR is therefore achieved if a constant force $F_N^{\text{target}} \in \mathbb{R}$ and constant translational velocity $v_T^{\text{target}} \in \mathbb{R}$ is maintained for the entire task.

A.2 Nominal Control Design

We use a classical motion control approach for the nominal controller (Lynch and Park, 2017). This control approach can be separated into path generation and motion control.

As the action space is defined in task space, the path r^{path} is defined via Cartesian positions and orientations. For the positions, we apply the Via-Points algorithm (Lynch and Park, 2017) using a third-degree interpolation method. The algorithm requires the specification of N via-points with their corresponding positions $\tilde{x}_n \in \mathbb{R}^3$ and directional vectors $\tilde{d}_n \in \mathbb{R}^3$ of the path, for all $n \in N$ via-points. The directional vectors $\tilde{d}_n$ are necessary hyperparameters for the Via-Points algorithm that represent the current path direction at the position of each via-point. Changing the directional vectors of a via-point leads to different curvatures of the resulting positional path. The position and the directional vectors then act as boundary constraints for the path, resulting in a well-defined analytical problem. The algorithm then outputs a piecewise function between each neighbouring via-point pair. For the orientation, we apply the spherical linear interpolation (SLERP) algorithm for quaternions (Shoemake, 1985). For SLERP, we specify the orientation of the end-effector $\tilde{\phi}_n \in \mathbb{R}^4$ at the positions of the N via-points. SLERP then results in the definition of piecewise functions between each orientation specification $\tilde{\phi}_n$ and $\tilde{\phi}_{n+1}$. After creation, the functions for the position and orientation are combined to form a coherent and continuous path function r^{path}. This path is then discretized into $M \gg N$ equally spaced control points $p_m^{\text{prior,path}} \in \mathbb{R}^7$.

The control points build the foundation for the motion control, which outputs the action a_t^{prior}. Given the current pose $p_t = (x_t, \phi_t)^\intercal \in \mathbb{R}$ of the robotic arm, we search for close control points that lie within a predefined radius $a_r \in \mathbb{R}$. Close control points can be grouped in the set P^{close}. From this set, we choose the control point $p^{\text{prior,ref}}$ that is furthest along the task direction. The chosen control point is then used as a reference for the controller. Figure A.1 visualizes this process.

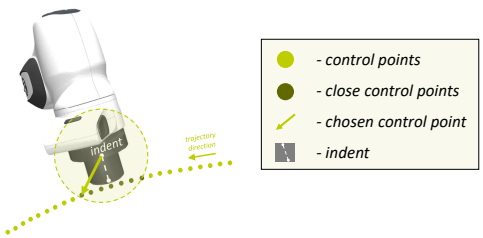

Fig. A.1. Generation of the next desired control point.

After determining the reference control point $p^{\text{prior,ref}} = \left(x^{\text{prior,ref}}, \phi^{\text{prior,ref}}\right)$, the action a_t^{prior} is computed using the following two equations for position and orientation

$$\Delta x_t^{\text{prior}} = x_t - x^{\text{prior,ref}},$$
$$\Delta \phi_t^{\text{prior}} = \phi^{\text{prior,ref}} \cdot \phi_t^{-1}.$$

Since the positional part of the path is defined directly on the workpiece surface, we shift the robot's position x_t to be located inside the end-effector. For this purpose, we define an indentation parameter Δx^{indent} that describes the absolute distance from position x_t to the end-effector's surface. Changing the indentation modifies the force behavior, with higher values leading to a stronger movement toward the workpiece.

To receive the full nominal controller action a_t^{prior}, $\Delta x_t^{\text{prior}}$ and $\Delta \phi_t^{\text{prior}}$ are concatenated into $\Delta p_t^{\text{prior}}$ and enhanced with constant impedance gains $K^{\text{prior}}, \zeta^{\text{prior}}$. The full nominal controller action is then defined as

$$a_t^{\text{prior}} = (\Delta p_t^{\text{prior}}, K^{\text{prior}}, \zeta^{\text{prior}}).$$

A.3 Details on the Reward Definition

At each time step t, the reward is a weighted sum of terms designed to encourage path-following, as well as velocity and force tracking such that

$$r_t = c_{c^\perp} r_{c^\perp,t} + c_{c\parallel} r_{c\parallel,t} + c_d r_{d,t} + c_v r_{v,t} + c_f r_{f,t}.$$

Here, $r_{c^\perp}$ and $r_{c\parallel}$ reward path-following in perpendicular and parallel path direction, r_d directional alignment, and r_v and r_f velocities and forces close to the respective targets. Each term is weighted with a constant $c_i \in [0, 1]$, and we choose these constants such that $\sum_i c_i = 1$. Since we normalize all reward terms, this choice of weighting factor makes it easy to interpret the relation between each reward term. In addition, we give a penalty r_{trunc} for episode truncation and a final termination reward r_{term} when the last via-point is successfully wiped.

We use two piecewise functions to transform an error value $e \in \mathbb{R}^+$ of the current state s_t into a reward. The function $l(e)$ is a purely linear mapping from error to reward and can be written as

$$l(e) = \begin{cases} 1 - \frac{e}{e_{max}}, & 0 \leq e \leq e_{max} \\ 0, & e > e_{max}. \end{cases}$$

The function $q(e)$ rewards values close to the targets quadratically and values far away from the targets linearly. The function is defined as

$$q(e) = \begin{cases} 1 - \frac{e}{2e_{mid}}, & 0 \leq e \leq e_{mid} \\ \frac{e_{max}^2 - e^2}{2(e_{max}^2 - e_{mid}^2)}, & e_{mid} < e \leq e_{max} \\ 0, & e > e_{max}. \end{cases}$$

Figure A.2 visualizes the two mapping functions.

Path following. The robot's path-following behavior is reinforced by using four reward terms

(1) the cross-error $e_{c\parallel}$ parallel to the path,
(2) the cross-error $e_{c^\perp}$ perpendicular to the path,
(3) the velocity error e_v,
(4) and the directional error e_d.

The cross-error is the distance from the current position x_t to the path r^{path}. It can be split into the parallel part $e_{c\parallel}$ and the perpendicular part $e_{c^\perp}$ according to the surface of the workpiece. Here, perpendicular means the part that

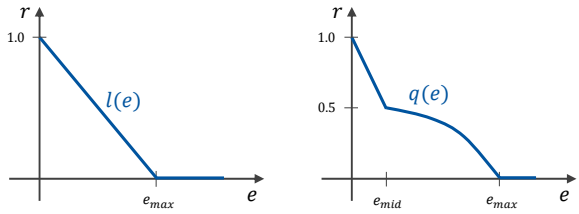

Fig. A.2. We define two functions to map an error to a reward. The left panel shows the linear transformation $l(e)$. The right panel shows the transformation $q(e)$ mapping outer values quadratically and values close to targets linearly.

aligns with the surface normal of the workpiece, essentially displaying the height deviation of the robot from the path. This way, we can define separate weightings for both cross-error parts. Given the position of the closest control point $x_m^{\text{prior,path}}$ according to the current position x_t, the parallel part of the cross-error can be computed as

$$e_{c\parallel}(s_t) = \left(x_{x,t} - x_{x,m}^{\text{prior,path}}\right),$$

while the perpendicular part is computed as

$$e_{c^\perp}(s_t) = \sqrt{\left(x_{y,t} - x_{y,m}^{\text{prior,path}}\right)^2 + \left(x_{z,t} - x_{z,m}^{\text{prior,path}}\right)^2}.$$

Note that this logic only applies to our task setup, where the x-direction always describes the lateral deviation from the path. Both errors are transformed into rewards $r_{c\parallel}$ and $r_{c^\perp}$ using transformation function $l(e)$.

In addition to that, we want to reward good velocity behavior according to the predefined velocity target v_T^{target}. Using the current velocity $\dot{p}$ from the state s, the velocity is computed as

$$e_v(s_t) = |\|\dot{x}_t\| - v_T^{\text{target}}|.$$

We transform this error e_v into a reward r_v by using the linear function $l(e)$.

Since the information about the movement direction is lost when using the norm of the velocity $\|\dot{x}_t\|$ for the reward r_v, we define an additional movement direction reward. For this, we make use of the current movement direction $d_t = \frac{\dot{x}_t}{\|\dot{x}_t\|} \in \mathbb{R}^3$ and the path direction $d^{\text{path}} \in \mathbb{R}^3$ of the closest control point. The path direction d^{path} can be computed for every control point as the normed distance vector to the subsequent control point. Given these two directions, we can compute the directional error as their alignment angle

$$e_d(s_t) = \arccos\left(d_t \cdot d^{\text{path}}\right).$$

The final direction reward r_d can be computed using the piecewise linear function $l(e)$. Splitting the velocity and the direction into two rewards again enables independent reward weighting.

Force Tracking. We define a force error based on the specified target force F_N^{target}. For this, the sensed force is normed $\|F_t\| = F_{N,t}$ and subtracted from the target

$$e_f(s_t) = |F_{N,t} - F_N^{\text{target}}|.$$

The force error e_f is transformed to the reward r_f using the linear-quadratic function $q(e)$. This transformation function emphasizes the reward for force values $F_{N,t}$ close to the target force F_N^{target}.

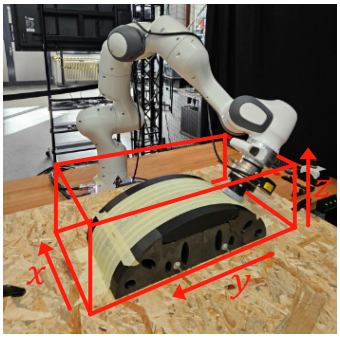

Fig. A.3. Positional safety limitations of the environment and the corresponding task space coordinate system.

Reward Weighting. Table A.1 shows the weights of the reward terms, while Tab. A.2 defines the boundary values for the piecewise error transformation functions. We distinguish between simulated and real hardware environments. We define wider reward boundaries for the hardware environment because this environment is generally more challenging. Both SAC and CHEQ are trained using the same set of parameters.

A.4 Safety Limits

In the following, we describe the safety limitations of the polishing environment. A violation of one of the limitations will lead to truncation of the episode. We use four types of limits: positional, orientational, velocity and force limits.

Positional limitations. In some applications, robotic arms are placed close to other facilities or stations of an automation process. Our positional limits restrict the arm movement inside a bounding box. We define the possible workspace of the robotic arm to be a cuboid, limiting the position $x = (x_x, x_y, x_z)$ of the end-effector close to the workpiece of the polishing process. The workspace cuboid is defined as

$$x_x \in [0.05, 0.15]\,[\mathrm{m}],$$
$$x_y \in [-0.23, 0.57]\,[\mathrm{m}], \text{ and}$$
$$x_z \in [0.0, 0.2]\,[\mathrm{m}].$$

Here, the restriction of the x_x-position is tight compared to the other axes since the target path r^{path} is defined in the yz-plane. The asymmetric boundaries of the x_y-position arise because the origin of the y-axis is defined by the initial state of the polishing path. The z-axis has its origin on the table, and the origin of the x-axis is at the border of the workpiece. Note that the positional limitations also passively restrict the robotic arm's joint displacements, so we do not define additional joint limits in our environment. Figure A.3 shows the positional limits.

Orientational limitations. In addition to positional limits, we also restrain the end-effector orientation. Combined with the positional limits, this aims to avoid collision with the surroundings, particularly with the workpiece. We speak of collision when the robotic arm has non-planned contact with the environment, i.e., if a part other than the end-effector has contact with the environment. In Euler angle representation [°], the orientational limits are defined as

$$\phi_x \in [-180, -110] \cup [110, 180],$$
$$\phi_y \in [-10, 10], \text{ and}$$
$$\phi_z \in [-10, 10].$$

Here, ϕ_x has a wider span compared to the other two rotation axes ϕ_y and ϕ_z since this rotation axis is used to perform the rotation movement of the end-effector.

Velocity limitations. To ensure safe movement of the robotic arm, we restrict the end-effector velocity $\dot{x}$. The limit of the velocity is defined as $\|\dot{x}\| < 0.5\,\mathrm{m\,s^{-1}}$. Anything above this threshold is considered dangerous for any setting of impedance gains.

Force limitations. We aim to avoid high-force interactions between the end-effector and the workpiece. To yield this, we limit the maximum contact force of the force sensor to a certain threshold $F_N < 25\,\mathrm{N}$. In addition to that, no contact with the environment is allowed when the end-effector position is below $z < 0.01\,\mathrm{m}$. Detecting a contact force in this position means the end-effector has undesired contact with the workpiece table.

A.5 Hyperparameters

This section details the hyperparameters used for our experiments.

SAC Hyperparameters. All SAC-specific hyperparameters for the simulated and hardware CHEQ agents, as well as the SAC variants, are described in Table A.3. To mitigate fluctuating λ^{RL} values on hardware, we choose a larger critic ensemble of 10 neural networks. For all algorithms, we include a random sampling phase for the first 15×10^3 steps where we sample the RL action uniformly random and do not update our agent. In this setting, we vary λ^{RL} between [0.2, 0.3] for the CHEQ agent.

CHEQ Hyperparameters. Table A.4 provides the hyperparameters specific to the CHEQ algorithm.

As the control frequency $f^{\text{RC,real}} = 20\,\mathrm{Hz}$ of the real hardware environment is lower than the control frequency of the simulation $f^{\text{RC,sim}} = 50\,\mathrm{Hz}$, it takes less episodic steps to perform the task successfully. We thus choose different horizon lengths for the two environments. This results in less accumulated return, which alters the values of the Q-networks. Hence, the Q-networks show lower values in the hardware environment than in the simulation. The final return in simulation is approximately half the simulated return. Lower values result in lower standard deviations or total uncertainty values. Thus, lower uncertainty limits must be chosen for the hardware setup to achieve the same behavior. Since we focus on safe exploration in our hardware experiments, we have increased the upper limit $u_{\max}$ to achieve safer exploration behavior through lower λ^{RL} values.

Nominal Control Hyperparamters.

Table A.5 provides the hyperparameters for the nominal controller for the simulated and real environment.

The spacing parameter describes the equidistant spacing between neighbouring control points used during path r^{path} creation. This setup consistently provides $\sim$ 50